WAKARU

Proposal · preliminary results · mechanistic interpretability
November 11, 2026

# A proposal for interpreting dense networks



Keira Wiechecki
Wakaru · kaw504@nyu.edu

Lamb Wakefield
Flock Capital · lamb@flock.holdings

Jade Zaslavsky
Wakaru · jadeofnameless@gmail

vmfunc
Flock Capital · celeste@linux.com

**Abstract**

Behavioral evaluation cannot settle what a model is trying to do, because a model can behave differently when it detects that it is being trained or evaluated. Auditing a model's goals requires identifying its internal variables, including concepts no one has thought to name (the ontology identification problem). Supervised probes and difference-of-means vectors currently outperform sparse autoencoder (SAE) features at concept detection and steering, but supervised methods only test concepts an experimenter already suspects. Unsupervised SAEs struggle in part because of the form of their features: each latent either fires or stays silent, feature absorption makes silence uninformative, and effective steering vectors instead contrast alternative values of a variable. We propose learning categorical variables directly by decomposing a representation space into competitive classifier heads, each of which assigns every input to one of $k$ entries associated with a learned atom. Forcing a head from one entry to another swaps one atom for another, an unsupervised analogue of a difference-of-means vector, though swaps between arbitrary partitions of the same space steer their own cells just as specifically. In preliminary experiments on SONAR and GPT-2, trained heads learn contrastive atoms, a 1900-bit hard code reduces reconstruction error by at least 33% relative to a 160-latent TopK SAE at the same bit budget (though wider or entropy-coded SAEs reach lower error), an 800-bit hard code on GPT-2 recovers more next-token loss than a $k_{\text{SAE}}$=32 SAE, and head-level interventions compose nearly additively. However, the learned variables are not yet identifiable: independent seeds reach the same four-digit reconstruction error while sharing no heads past the first layer, and input-space steps flip other heads as often as random steps of the same norm. The one exception is a block of slow heads trained on adjacent sentence pairs, which reproduces across three seeds, though not yet head by head (matched NMI 0.41 vs. a 0.005 null). Because an audit or a learned objective requires features that remain consistent across retraining, identifiability rather than reconstruction fidelity is the binding constraint, and our proposed work begins with paired-view objectives.

## 1 Introduction

Ideally we would like mechanistic interpretability to provide guarantees about the computational process used by a model. We would like features to be

1. causal
2. disentangled

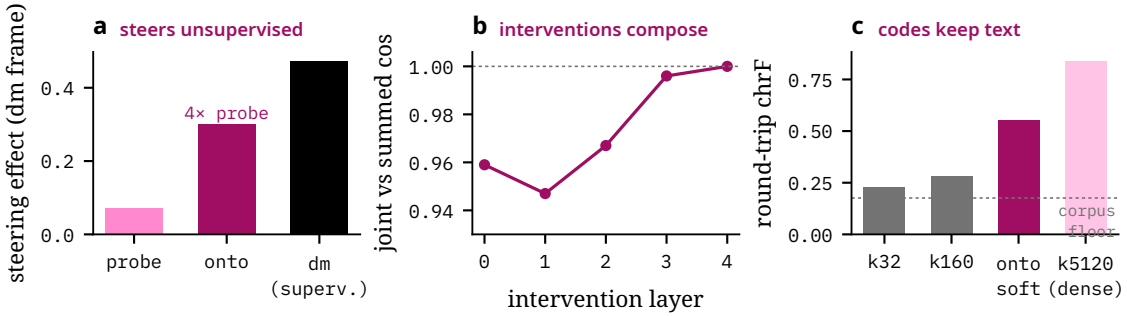



**Figure 1** Steering and reconstruction in the softmax Ontologizer. (a) Forcing an entry moves decodes along the supervised difference-of-means direction 0.64× as far as supervised steering itself and 4× as far as probe directions (Table 4). (b) Joint two-head interventions match the sum of the individual interventions, and match it exactly in the final layer. (c) Sparse SAE reconstructions lose most of the text before any intervention is applied (round-trip chrF 0.23–0.28 against a 0.18 corpus floor), whereas the soft code preserves it (0.55); the dense $k_{\text{SAE}}$=5120 SAE preserves more text but is not sparse.

3. closed-form

However most existing techniques, such as sparse autoencoders (SAEs), produce features that can perform arbitrarily badly on these desiderata. As a dictionary grows, features split into narrower ones [8], and a narrow feature can absorb a broader one so that the broader feature fails to fire where it applies [13], breaking disentanglement. SAE features can't usually be statically analyzed (though bilinear SAE variants can be). Sparsity trades off against reconstruction fidelity, limiting the extent to which features can be used for steering.

Arguably the simplest interpretability technique is a linear probe, and the simplest control technique is a steering vector. On current benchmarks both beat features from unsupervised techniques such as SAEs, with probes ahead on concept detection and the difference of means ahead on steering [56], but both require contrastive datasets built around a concept someone has already named [9]. In the spirit of MoEfication [59], we propose a technique based on decomposing a representation space into a set of contrastive classifiers. Each entry corresponds to a representative vector (an atom). Inputs are encoded as a series of classifier assignments and reconstructed from the atoms those assignments select. This has several advantages over SAEs:

1. It effectively gives steering for free: changing a head's entry is a steering move.
2. Outputs are bounded.
3. The representative vectors admit static interpretation in principle, though making this work in practice remains an open problem (Section 6.9).
4. Feature space is structured.

Figure 1 summarizes the steering behavior of the soft model, and Section 6 reports our preliminary evaluation, including where the results do not support these hypotheses.

## 2 Assumptions

This proposal assumes:

- **Steering is inherent to the training objective.** The dictionary vectors must also be high quality steering vectors; interpretability and steering are treated as the same problem.
- **Weights are themselves interpretable** (for certain architectural choices).
- **Clustering rather than decomposition.** Representations are grouped into mutually contrastive categories rather than decomposed into independently firing features.
- **Categorical structure is imposed on features.**
- **Interpretability gets a cheap forward pass.**
- **Concepts compose linearly.** Concepts must be additive to be useful.
- **Dictionary learning is all you need**, in a nonstandard form: the dictionary forms a basis for the representation rather than a target or a discrete codebook.
- **Representations are not themselves quantized.** Quantization comes from sharpening the classifications over that basis. When a soft relaxation is trained from scratch, however, the decoder can learn to invert the soft probabilities instead of quantizing [1, 45]; we address this with straight-through hard selection (Section 6.3).
- **Interpreting a well trained model is the same problem as interpreting the data and training tasks.**

## 2.1 Pivotal assumptions

Two of these are pivotal: some version of the linear representation hypothesis [46, 47] is true, and latent space is in some way "natural".

## 2.2 Concerns

**Contrastive classifiers might not be expressive enough** This is what the reconstruction benchmarks test. A soft code pays a large reconstruction penalty for its contrastive structure, whereas a pure 1900-bit hard code outperforms `TopK` SAEs at the same bit budget, though not wider or entropy-coded SAEs (Section 6.3).

**Heads might not be identifiable** We can only assign a stable meaning to a head if similar heads emerge across training runs. Past the first layer, they do not: two seeds reach identical reconstruction error while their head partitions agree barely above chance (Section 6.6). The only heads that reproduce across seeds so far are a block of slow heads trained on adjacent sentence pairs, and they reproduce as a block rather than one by one (Section 6.7). Identifiability is currently the main bottleneck for this proposal, consistent with results showing that unsupervised learning from marginal data alone cannot recover latent factors without matching inductive biases on the model and data [38, 26]. This motivates cross-seed selection [20, 2] and paired-observation objectives [28, 34, 39, 32, 60] (Section 7).

**Steering could push it too far off manifold** This is an open problem in activation steering. In robot control, a frozen diffusion policy can be steered through its input noise rather than its outputs, using a reinforcement-learned noise policy to keep actions within the denoiser's training distribution [53]. The analogous approach here is steering through the code by selecting entries and decoding, rather than adding arbitrary vectors in activation space. Even where off-manifold drift remains an issue, classifier heads provide a testbed for steering methods: for instance, whether RL against a head's classification score converges faster than paired steering or PCA vectors.

**Heads could be entangled** Interventions applied directly to head assignments compose cleanly, but stepping in input space flips other heads as often as a random step of the same length, and stacking residual layers roughly doubles this interference (Section 6.4).

**Representations could be too soft/dense/high entropy** This is highly correlated across all feature-based interp (SAEs, transcoders [21], J-space). Dividing representations across orthogonal categorical subspaces makes it possible to interpret the conceptual basis vectors even if the representations are dense mixtures. In our softmax models, however, taking the argmax of a soft code yields poor reconstructions (Section 6.1), which motivates training hard codes directly.

**Entries could be too polysemantic** Orthogonality and categorical constraints give a theoretical inductive bias to monosemanticity. As long as they have decent explanatory power we can do eigendecomposition; polysemanticity should decrease with parameter count, and there are several directions to increase parameters (width, head count, layers). An initial short-training scaling sweep found no increase in head coherence with parameter count (working notes), though full-length scaling retrains have not yet been run.

**Interpreting entries could be otherwise intractable** This would imply the universality hypothesis is false and is highly correlated with intractability of other interpretability agendas. Submodules could be interpretable even if features aren't.

**Training could be intractable** Hard categorical codes can be difficult to optimize, and our straight-through models require the auxiliary stabilizers derived in the working notes (Section 6.3).

**Doesn't converge in the limit** Collapse in one layer would propagate down the stack. Upper-layer heads do freeze when the batch-mean term $\mathrm{KL}_m$ (Eq. (4)) is omitted, whereas including $\mathrm{KL}_m$ prevents late freezing (working notes).

**Not orthogonal in the limit** The support overlap penalty $\omega$ (Section 5.1.3) penalizes heads that share dictionary coordinates, but summed models do not form a coordinate partition in practice: atoms within a head spread apart, yet heads continue to share coordinates, and penalizing $\omega$ on the dictionary weights pushes atom energy into the decoder's null or near-null space while leaving decoded overlap above chance (Section 6.2). A static analysis of the dictionary therefore cannot assume that summed heads occupy disjoint subspaces. Concatenated heads enforce disjoint dictionary blocks by construction and outperform direct atoms on GPT-2 once each head's slice is wider than its $k$ entries (Section 6.8).

**Might not generalize out of distribution** The same concern applies to every feature-based technique. Closed-form interpretation of the dictionary is the mitigation: a statically interpreted basis does not depend on the input distribution, though the classifications over it do.

**Inference too expensive** Each layer computes all $h \cdot k$ classifier logits densely per sample (Section 5.1), and a bilinear classifier costs roughly twice as much as a linear encoder of the same width, so the architecture does not reduce forward-pass compute relative to an SAE of width $l\,h\,k$. Reducing encoder compute would require routing each input to a subset of heads, similar to Switch SAEs [42]. During training, deep supervision [37] also requires a decoder pass at every layer.

**Autointerp doesn't work on parameter decodes** In our preliminary experiments, decoding entry directions from parameters alone fails to produce usable descriptions (Section 6.9), which contradicts the static-interpretability claim.

# 3 Motivation

## 3.1 Threat Model

**The ontology identification problem.** We can't yet reliably look inside a model and draw conclusions about its goals or world model. Interpretability tools have recovered a hidden objective in an auditing game with a planted behavior, but a semantic search of the training data surfaced the same evidence without inspecting the model at all [41]. This is a major limitation on almost all proposed alignment agendas, because behavioral testing alone cannot settle what a model is optimizing for: a model can behave differently when it infers that it is being trained or evaluated, and when that context comes from training documents rather than the prompt, the gap can persist without any visible chain of thought [25]. Although not every model exhibits this gap [52], the broader point that aligned behavior during testing need not predict behavior after deployment is standard [6, Ch. 8].

**Inner misalignment** We don't currently know how to specify a robust goal for a model. Because goal space is large, we should expect goal misgeneralization as the default outcome.

**Limits of natural abstractions.** Networks trained on the same task can converge on the same algorithm across architectures while differing, even across random seeds, in which features implement it [14]. Similarly, in language models, SAE features from a Transformer and a Mamba trained on the same corpus match nearly as well as those from two Transformer seeds [54]. Both results support a weak form of the universality hypothesis—the claim within the natural abstraction hypothesis that diverse cognitive systems learn similar high-level abstractions [12]—which, if true, would provide a convenient shortcut for auditing models.

Natural abstractions could be considered the set of concepts we could use to communicate with aliens. They should consistently emerge through unsupervised learning (in the limit given some constraints). If any neural network is considered a fuzzy generalization of a finite state machine, natural abstractions are the latent states.

Unfortunately, the abstractions a model learns seem highly sensitive to choice of training data, and even to the random seed with the data fixed [14], and are thus not necessarily universal in the limit. [1] Assuming that model capabilities will continue to primarily be driven by access to more data and more modalities, we cannot expect the abstractions learned by a weak model to remain robust in a more powerful model.

Even if we could fully characterize a model's state space in terms of natural abstractions, that still doesn't mean we can reconstruct the latent FSM, as the transition rules can be arbitrarily complex.

## 3.2 Theory of change

Conditional on the underlying model of ontology being true, the techniques proposed here would be a substantial interpretability advance. The agenda also has a case for counterfactual impact: its theoretical basis consists of nonobvious applications of obscure techniques from niche fields of bioinformatics, and similar research is unlikely to be done soon otherwise.

[1] The same applies to those that seem most natural to humans.

**What a solution would look like.** Mechinterp plausibly offers the best path to a robust alignment solution despite criticisms. The case rests on

1. **decomposability** Mechinterp allows alignment to be broken into manageable subproblems.
2. **parallelizability** Decomposable subproblems can be worked on independently.
3. **robustness** Mechinterp offers direct causal explanations of model behavior.

The most straightforward mechinterp alignment solutions are

1. eliciting latent knowledge (ELK) Directly inferring a model's ontology.
2. mechanistic anomaly detection (MAD) Detecting when a model has "unusual" thoughts.

Both of these can be considered ontology identification subproblems. MAD requires mapping between the model's activations and the model's ontology. ELK requires mapping between the model's ontology and human ontology. These are hard but not intractable problems. Recently, developmental interpretability (devinterp) techniques have identified distinct "developmental stages" during training, each marked by a shift in internal structure and behavior such as learning bigram statistics or forming induction heads [27]. This suggests, at least in small transformers, that training consists largely of the network grokking [44] discrete concepts. While these results are promising for MAD, locating these emergent capabilities in the model's activations remains an open problem.

Interpretability glosses broadly as "let humans understand AI". The alternative—getting AI to understand humans better—scales badly: making the model smarter leads to subtler and subtler issues with larger and larger effects. Between the two sits scalable oversight, in which models help humans supervise other models; humans paired with an unreliable dialog assistant already outperform both the assistant and unaided humans [7], and interpretability provides another signal such oversight can use. Moreover, what we're doing is not just letting humans understand AI. Because an LLM reflects humanity from its training data, high quality conceptual interpretability provides machine-comprehensible mathematical representations of every concept that humanity cares about. We work on interp for the usual corrigibility reasons, and also because outer alignment needs a stable target to attach values to: a goal defined in terms of a model's internal concepts must continue to point at those concepts as the model's representations are updated [6, Ch. 12]. This agenda attacks both problems, though our preliminary results show how hard concept stability is even at small scale: past the first layer, learned heads do not survive a change of random seed (Section 6.6).

**Macro-scale strategy.** Broadly, the goals are to

1. formalize the notion of a model's latent concept space
2. distill a model's latent concept space
3. enforce decomposability of latent concept space
4. enforce robustness of existing concepts as the dimensionality of the latent concept space increases

A prerequisite for all of these is the ability to express a metaontology which can identify latent concepts in activation space.

# 4 Background

Mechanistic interpretability currently can do supervised steering well or unsupervised steering poorly. The Google DeepMind mechanistic interpretability team has pivoted from ambitious reverse-engineering to pragmatic interpretability judged on proxy tasks, and has deprioritized SAE research:

SAE probes did not generalize better out of distribution, and SAEs did not beat a steering vector built from a single contrastive pair of prompts at suppressing evaluation awareness [43]. On current benchmarks, supervised difference-of-means vectors steer better than SAE latents, though still worse than prompting [56]. However, supervised methods only find concepts for which an experimenter already has examples, and a direction optimized to produce a desired output change need not reflect how the model normally computes: a targeted subspace intervention can instead route through a dormant pathway [40]. The dominant unsupervised paradigm is the sparse autoencoder (SAE). A model's hidden state is approximated as a finite state machine consisting of discrete features. The first implementations used a `ReLU` activation and sparsity loss [17, 8], though more recent implementations have used a `TopK` activation function without a sparsity loss [24].

## 4.1 Common problems with SAEs

The motivation is to decompile a model to a finite state machine (FSM). The limitation is that such an FSM is only as interpretable as an enumeration of all possible states; even a mixture of two simple processes (one constant and one a fair coin) requires infinitely many states to track which component is active, because the structure lies in how the components overlap [16].

**Features are not disentangled.** A representation is disentangled when changing a single underlying factor of variation changes a single factor of the representation [38]. A persistent problem with SAEs is feature absorption [13]: when a narrow feature implies a broader one, sparsity rewards folding the broad feature's direction into the narrow latent and leaving the broad latent off, so a seemingly monosemantic latent fails to fire on subset cases (for instance, a "starts with S" latent that stays silent on "short"). A related failure mode, feature composition, merges frequently co-occurring concepts into a single latent. Matryoshka SAEs [10] mitigate both issues by training nested dictionary prefixes to reconstruct the input independently, encouraging early latents to learn broad features and later latents to learn narrower ones. Tree SAEs [11] enforce hierarchy explicitly by gating child latents on assigned parents.

**Features are not cleanly causal.** Sparsity trades off against reconstruction fidelity [24]. Using SAE features for steering often produces inconsistent, context-sensitive effects: adding a decoder direction shifts logits differently across contexts and perturbs unrelated downstream features. While steerability varies across features and can be partially predicted from decoder-direction crowding and direct logit effects, the best predictors vary across models and dictionaries [19].

**Features are not closed-form.** A major concern is how well an interpretability technique generalizes out of distribution. Ideally we would like to infer behavior by looking solely at model weights. Bilinear MLPs make this partly possible because the interaction matrix for any output direction can be eigendecomposed without data, though in deep networks the target output directions still depend on data-driven SAEs [49].

**Forward passes are expensive.** Efficiency is also a problem. Although the sparsity constraint means that only a handful of features are active for a given input, the entire feature space needs to be computed for each sample [24, 42]. Mixture-of-experts (MoE) SAE variants reduce compute costs by dividing feature space between smaller submodules. Examples include Switch SAEs, which route each input to one expert [42], and Scale SAEs, which activate several experts per input to reduce feature duplication across experts [57].

Less explored is the effect of the structure imposed by the MoEfied architecture on the interpretability of feature space. A common approach to scalably interpreting features is to have an LLM generate an

explanation from activating examples and score how well an LLM judge can distinguish activating from non-activating contexts using that explanation alone [48] (in our experiments, the same model both generates and scores explanations). With an unstructured feature space, it quickly becomes intractable to interpret all combinations of features this way, whereas structured expert routings can be analyzed one module at a time. The ReLU feed-forward layers of a trained transformer can themselves be partitioned into experts of co-activating neurons, retaining over 95% of task performance while activating only 10–30% of the parameters [59]. Language models also appear to represent categorical concepts as simplices whose vertices correspond to category values, with hierarchically related concepts placed in orthogonal subspaces and differences between vertices acting as contrastive steering vectors [47]. This suggests that MoEfication into classifiers can expose latent contrastive steering vectors, since swapping one class entry for another applies the difference between their vectors. Similarly, penalizing the cosine similarity between each SAE decoder direction and its nearest neighbor reduces feature absorption by roughly two thirds relative to BatchTopK SAEs at a small cost in explained variance and unchanged autointerp scores [35]; whereas that penalty is unstructured, the Ontologizer enforces competition within heads and seeks orthogonality between them.

Existing attempts to route activations to expert SAEs focus on routing similar inputs to the same expert [42], and score an expert as specialized when its features resemble each other and not other experts' [57]. Contrastive learning wants the opposite: training on things at opposite ends of a shared axis. We don't just want to find similar features, we want to find contrastive vectors in feature space that correspond to what a model considers a natural category.

## 4.2 Disjoint support

In the concatenated-head variant, each head writes to its own slice of dictionary coordinates, so atoms from different heads have disjoint supports by construction. For non-negative atoms, disjoint support is equivalent to orthogonality (Eq. (6)), whereas signed vectors can be orthogonal through cancellation while sharing every coordinate. When heads are summed across the full dictionary space and encouraged to separate only through a penalty, a coordinate partition fails to form: no trained summed model partitions its dictionary coordinates between heads, and penalizing support overlap $\omega$ on GPT-2 reduces dictionary-space overlap only by shifting atom energy into directions that the decoder shrinks or discards (Sections 5.1.3 and 6.2). Enforcing disjoint support by concatenation affects each of the four SAE failure modes above differently; in every case, the guarantee holds between heads within a single layer's dictionary space, before the shared linear decoder projects those blocks into the output space.

**Disentanglement.** Feature absorption occurs when a general latent turns off in the presence of a narrower latent whose decoder direction absorbs the general one. Classifier heads prevent the first step because every head must select an entry on every input and cannot simply turn off; disjoint dictionary blocks prevent the second step in dictionary space because one head's atoms have zero weight on every other head's coordinates. Feature composition (merging two independent factors into a single head) is discouraged rather than ruled out. Because $h$ heads with $k$ entries define $k^h$ combined codes from $hk$ atoms, independent factors can be represented by separate heads. Theoretical results for non-negative representations with activity and weight regularization show that two factors take disjoint supports when their joint distribution covers a broad enough product range [55, 18]. Our learned SONAR codes satisfy the analogous empirical coverage condition: among entry pairs from two heads that would occur at least ten times under independence, at most 0.24% have zero count in any layer. However, without activity regularization the summed models do not partition their coordinates, whereas concatenated heads enforce the partition directly. Even so, unsupervised reconstruction of marginal

data cannot guarantee that individual heads align with true generative factors without matching inductive biases [38], as our cross-seed tests confirm (Section 6.6); geometrically, too, a Minkowski sum of convex hulls, the set a layer of heads can reach, does not determine its summands [23].

**Causality.** One source of steering collateral in SAEs is crowding among decoder directions [19]. With concatenated heads, atoms from different heads never overlap in dictionary space: changing one head's entry modifies only that head's coordinate block, orthogonal to the writes of all other heads in the same layer, so each head's contribution can be read directly from its slice. Summed heads also compose additively by linearity, but because their dictionary writes overlap, a single head's contribution cannot be isolated from a layer's pooled vector. Disjoint dictionary blocks do not, however, prevent collateral when stepping in input space: re-encoding a stepped input flips as many other heads as a random step of the same norm, both because hard-code heads sit near their decision boundaries and because downstream residual layers reclassify the modified residual (Section 6.4).

**Closed form.** Interpreting a head statically from its dictionary weights requires each head to occupy its own coordinates across all inputs rather than merely having low average overlap on a dataset (Section 5.1.3). Summed models violate this condition, with each head's usage spanning 52–98% of dictionary coordinates. Concatenated heads satisfy it by construction, and when each head's decoder block has full column rank, every direction within a head's slice maps nontrivially to the output (unlike the degenerate solutions produced by penalizing $\omega$). This structural property is necessary for weight-level interpretation, though not sufficient on its own: decoding entry directions through the SONAR decoder has not yet produced usable descriptions (Section 6.9).

**Cost.** Disjoint support does not reduce encoder compute. Concatenating heads partitions the dictionary width $e$ across $h$ heads, so each head stores $k$ atoms of dimension $e/h$ ($h$ times fewer dictionary parameters than a summed layer of width $e$), and a hard forward pass in the dictionary reduces to $h$ slice lookups. However, the classifier still evaluates all $hk$ logits per sample, just as an SAE evaluates all of its latents. If each head's slice is too narrow, concatenation also hurts reconstruction: on GPT-2 layer-8 residuals with non-negative atoms, constant temperature, and $d_{\text{head}}$=$k$=32 (where $k$ orthogonal non-negative atoms require $k$ coordinates), concatenated heads had the highest held-out error among the orthant variants ($\text{FVU}_w$ 0.416 vs. 0.154–0.293; working notes). Giving each head extra dimensions ($d_{\text{head}}$=48 $>$ $k$=32) and annealing the temperature reduces FVU to 0.186 (0.164 at $k$=64), outperforming direct atoms at 0.195 (Section 6.8).

# 5 Methods

## 5.1 Architecture

We propose a method that encodes samples as classifications mapped to contrastive vectors. This directly constrains a model's representations, and classifications can natively be used for steering.

The model (Figure 2) consists of $l$ `DictEnc` layers writing into a shared residual accumulator, decoded by a single linear map. Each layer (Figure 3) is composed of a classifier followed by a dictionary. The classifier splits its input into $h$ independent *assignments*, each a distribution over one head's $k$ *entries*, and the dictionary maps each entry to a representative vector (an *atom*). Each layer writes the assignment-weighted sum of its atoms to the accumulator; after each layer the accumulator is decoded and the residual $\mathbf{x} - \text{dec}(\mathbf{r}_\ell)$ is passed to the next layer, where $\mathbf{r}_\ell$ accumulates the $\ell$ layers before it. Deep supervision [37] constrains every prefix of the stack to reconstruct the input.

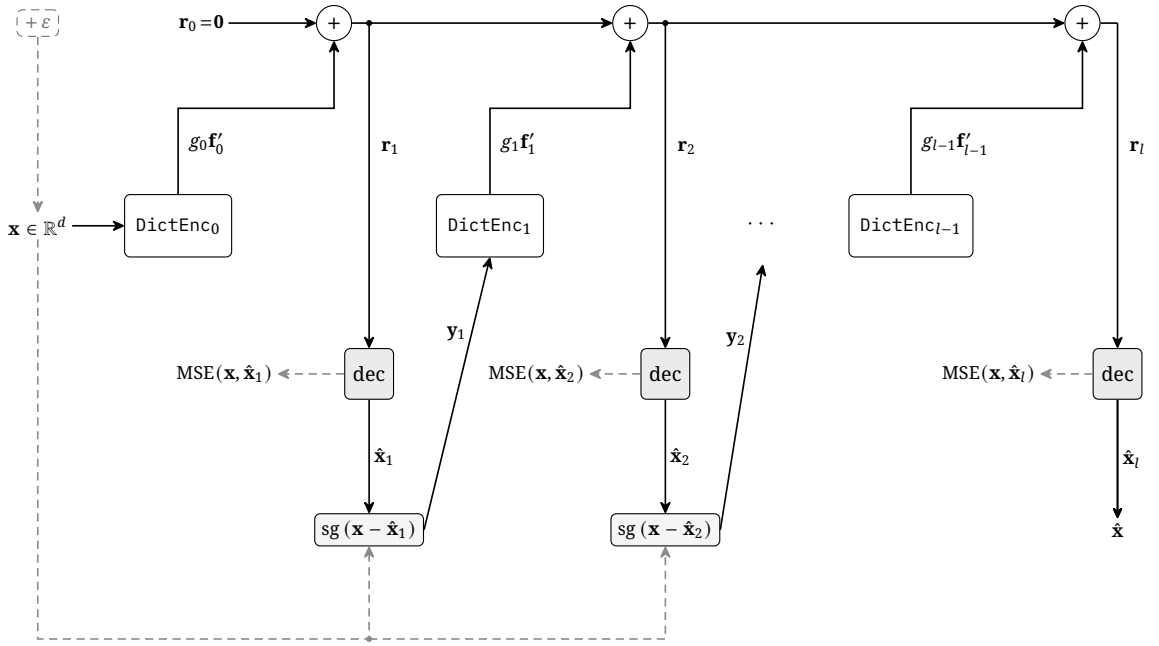



**Figure 2** **`Ontologizer` architecture.** An `Ontologizer` consists of $l$ stacked `DictEnc` layers. Each returns a pooled feature vector $\mathbf{f}'_\ell \in \mathbb{R}^e$, which is written, scaled by its input's gain $g_\ell$, to a residual accumulator $\mathbf{r}$. Layers are indexed $\ell = 0, \ldots, l-1$ and accumulator states by the number of layers applied: $\mathbf{r}_0 := \mathbf{0}$ and $\mathbf{r}_{\ell+1} = \mathbf{r}_\ell + g_\ell \mathbf{f}'_\ell$, so layer $\ell$ reads state $\ell$ and writes state $\ell + 1$; a stop-gradient variant (sg on $\mathbf{r}_\ell$, available but off in every model reported here) would prevent gradient communication between layers. dec is a linear decoder with shape $\mathbb{R}^{d\times e}$, shared by all layers. Deep supervision makes the code a successive refinement, coarse-to-fine in distortion as in residual quantizers [31, 4, 58] and nested-prefix SAEs [10]: the component encoded by previous layers is inaccessible to later layers. After $\ell$ layers, $\hat{\mathbf{x}}_\ell = \mathrm{dec}(\mathbf{r}_\ell)$, and the component left unexplained is $\mathbf{y}_\ell = \mathbf{x} - \hat{\mathbf{x}}_\ell$, so $\mathbf{y}_0 = \mathbf{x}$. Layer $\ell$ classifies the shape of its input, $\mathbf{u}_\ell = \mathbf{y}_\ell / \mathrm{L2}(\mathbf{y}_\ell) \oplus [1]$, and scales what it writes by the gain $g_\ell = \mathrm{L2}(\mathbf{y}_\ell)$. Normalizing prevents exploding gradients from repeated application of bilinear layers, each of which scales its input by $\|\cdot\|^2$; the constant coordinate keeps the sign of $\mathbf{y}_\ell$, which the bilinear form would otherwise lose. $\mathrm{MSE}(\mathbf{x}, \hat{\mathbf{x}}_\ell)$ is computed separately for each prefix $\ell = 1, \ldots, l$. The input-noise hook drawn on $\mathbf{x}$ is disabled in every model reported here.

Because entries within a head are mutually exclusive rather than independent, each head acts as a contrastive set: a $k$-way classifier whose entry swaps are steering moves. An auxiliary between-head penalty is intended to push each head onto a disjoint set of dictionary coordinates (Figure 4), though in practice summed models do not form a coordinate partition (Section 5.1.3).

**Classifier.** Each `DictEnc`'s classifier is a multihead bilinear map (`BilinearBlock`) with weights $[\mathsf{W}, \mathsf{V}] \in \mathbb{R}^{2\times h\times k\times d}$ ($d + 1$ with the constant coordinate of Eq. (9)). It reads the normalized shape $\mathbf{u}_\ell$ of the layer's input (the residual left by previous layers, or $\mathbf{x}$ at layer 0), written $\mathbf{x}$ below for a generic input, and returns logits $K \in \mathbb{R}^{h\times k}$:

$$\texttt{BilinearBlock}([\mathsf{W}, \mathsf{V}], \mathbf{x}) := \left(\sum_{q=1}^{d} W_{\cdot\cdot q}\, x_q\right) \odot \left(\sum_{q=1}^{d} V_{\cdot\cdot q}\, x_q\right). \tag{1}$$

Each logit is a quadratic form in $\mathbf{x}$, the low-rank form of bilinear classifiers [50, 33], so a classifier admits closed-form eigendecomposition in the manner of bilinear MLPs [49].

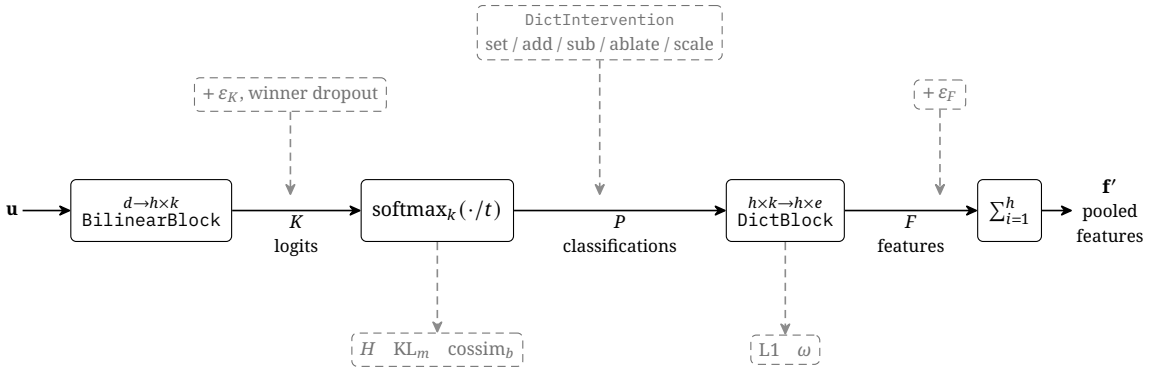



**Figure 3** **`DictEnc` architecture.** Dashed lines above indicate perturbations to the data, including noising and causal interventions. Dashed lines below indicate statistics used as tuneable auxiliary loss functions. Each `DictEnc` consists of a multihead bilinear classifier (Section 5.1.1) and a multihead dictionary (Section 5.1.3). Noise $\varepsilon_K$ and $\varepsilon_F$ and winner dropout regularize training (Section 5.1.5). Assignments $P$ are softmax$_k(K/t)$, or its straight-through argmax (Section 5.1.2); interventions act on $P$. From $P$ we collect per-sample entropy $H$, between-sample similarity cossim$_b$, and the divergence of the batch-mean assignment from uniform KL$_m$ (Eqs. (2)–(4)). A `DictBlock` maps $P \in \mathbb{R}^{h \times k}$ to features $F \in \mathbb{R}^{h \times e}$ (Eq. (5)); from $P$ and the atoms we collect the support overlap $\omega$ (Eq. (7)), and from $F$ the sparsity penalty L1. $F$ is summed over heads to give pooled features $\mathbf{f}' \in \mathbb{R}^e$, which are added to the accumulator $\mathbf{r}$.

**Assignments and selection.** Soft assignments are given by $P = \text{softmax}_k(K/t)$, with the temperature $t$ annealed during training to sharpen the distributions. When trained from scratch with soft assignments, however, the model continues to rely on the full probability vector rather than its discrete argmax, since the linear decoder can invert a soft relaxation of quantization [1, 45] (Section 6.1). To train a genuinely discrete code, the *straight-through* selection rule selects the one-hot argmax in the forward pass and uses softmax$(K/t)$ as a surrogate gradient in the backward pass, following the categorical straight-through estimator [5, 29]. A hard code transmits $l\,h \log_2 k$ index bits per sample, with the per-layer gains $g_\ell$ as its only continuous values.

From the assignments $P_n$ of each sample $n$ we collect per-sample entropy, in bits,

$$\text{entropy}_k(P)_i := -\sum_{j=1}^{k} p_{ij} \log_2 p_{ij}, \qquad H := \mathbb{E}_n\, \mathbb{E}_i \left[ \text{entropy}_k(P_n)_i \right], \tag{2}$$

between-sample similarity, the mean absolute cosine of the per-head features over pairs of samples, with $\cos\angle(\mathbf{a}, \mathbf{b}) := \langle \mathbf{a}, \mathbf{b} \rangle / (\|\mathbf{a}\| \|\mathbf{b}\|)$ the cosine of the angle between two vectors, evaluated through each head's dictionary Gram $G_i = |D_i||D_i|^\top$,

$$\text{cossim}_b := \mathbb{E}_{n,n'} \left| \cos\angle(\mathbf{f}_n, \mathbf{f}_{n'}) \right|, \qquad \langle \mathbf{f}_n, \mathbf{f}_{n'} \rangle = \sum_{i=1}^{h} \mathbf{p}_{ni}^\top G_i\, \mathbf{p}_{n'i}, \tag{3}$$

and the divergence of the batch-mean assignment from uniform,

$$\text{KL}_m := \mathbb{E}_i \left[ \log_2 k - \text{entropy}_k(\mathbb{E}_n[P_n])_i \right]. \tag{4}$$

KL$_m$ anchors each head's mean assignment to uniform, which keeps every entry in use.

**Dictionary.** A `DictBlock` with weights $\mathsf{D} \in \mathbb{R}^{h \times k \times e}$ maps assignments to per-head features $F \in \mathbb{R}^{h \times e}$, whose row $\mathbf{f}_i$ is head $i$'s output,

$$\mathbf{f}_i := \sum_{j=1}^{k} p_{ij}\, \mathbf{a}_{ij}, \qquad \mathbf{a}_{ij} := |\mathsf{D}_{ij\cdot}|. \tag{5}$$

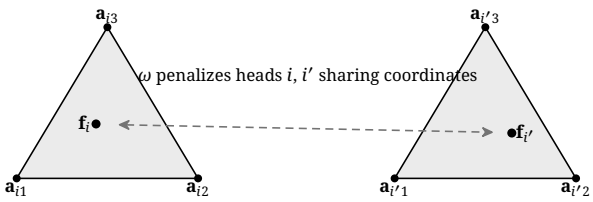



**Figure 4** `DictBlock` **head geometry** for $h$=2; $k$=3. The output space of a head $i$ is a simplex with its atoms $\mathbf{a}_{ij}$, the rows of $|D_i|$, as vertices. The assignment $P$ is a soft choice among mutually contrastive categories rather than a set of independently firing features, and because $p_{ij} \in [0, 1]$ the output lies within the convex hull of the head's atoms. Because $|\mathsf{D}| \geq 0$ and $P \geq 0$, $F \geq 0$, so $\cos \angle(\mathbf{f}_i, \mathbf{f}_{i'}) \in [0, 1]$, and driving the support overlap $\omega$ to zero encourages the heads to partition the coordinates of $\mathbb{R}^e$ between them (Section 5.1.3).

Atoms are non-negative and assignments lie on the simplex, so features are strictly additive and each head's output lies in the convex hull of its atoms (Figure 4). Summed over heads, a layer's output ranges over the Minkowski sum of these hulls, the form that the Minkowski Representation Hypothesis proposes for transformer activations [23].

**Disjoint support** For non-negative vectors the inner product is a sum of non-negative terms, so it vanishes if and only if the supports are disjoint,

$$\langle \mathbf{u}, \mathbf{v} \rangle = 0 \iff \operatorname{supp}(\mathbf{u}) \cap \operatorname{supp}(\mathbf{v}) = \emptyset. \tag{6}$$

Between heads, orthogonality on non-negative atoms is therefore a support-partition condition rather than a rotation constraint. The support overlap $\omega$ measures the mean cosine between heads' usage-weighted coordinate profiles:

$$\boldsymbol{\pi}_i := \sum_{j=1}^{k} \bar{p}_{ij}\, |\mathbf{a}_{ij}|, \qquad \omega := \mathbb{E}_{i \neq i'} \frac{\langle \boldsymbol{\pi}_i, \boldsymbol{\pi}_{i'} \rangle}{\|\boldsymbol{\pi}_i\| \|\boldsymbol{\pi}_{i'}\|}, \qquad \bar{\mathbf{p}}_i := \mathbb{E}_n\, [\mathbf{p}_{ni}], \tag{7}$$

which vanishes if and only if no two heads share an active coordinate. Because softmax assignments give every entry positive weight, two heads that are orthogonal on even a single sample must have disjoint supports across all $k^2$ pairs of their atoms, making the partition a weight-level property of $\mathsf{D}$ that holds for all inputs. In the concatenated-head variant, each head writes to a separate coordinate slice by construction and $\omega$ is identically zero.

This design distinguishes between-head and within-head geometry: disjoint support assigns each head its own coordinate block, while the $k$ atoms *within* a head act as contrastive alternatives spread across that block. When heads are summed rather than concatenated, $\omega$ is only a soft penalty, and empirically the within-head contrast emerges while a between-head coordinate partition does not (Section 6.2).

**Decoder.** Each layer's pooled output $\mathbf{f}'_\ell = \sum_{i=1}^{h} \tilde{\mathbf{f}}_i$ is scaled by its input's gain $g_\ell$ and added to the accumulator, which is projected back to the input space by a single unbiased linear decoder shared across all layers:

$$\mathbf{r}_\ell = \sum_{\ell'=0}^{\ell-1} g_{\ell'}\, \mathbf{f}'_{\ell'}, \qquad \hat{\mathbf{x}}_\ell = W_{\text{dec}} \mathbf{r}_\ell, \qquad W_{\text{dec}} \in \mathbb{R}^{d \times e}. \tag{8}$$

The residual $\mathbf{y}_\ell$ entering layer $\ell$ is stop-gradiented so that earlier layers cannot pass a communication code through the residual instead of reducing reconstruction error. It is decomposed into a scalar gain

$g_\ell$ and a normalized shape vector $\mathbf{u}_\ell$ with a constant coordinate appended, which forms the input to layer $\ell$'s classifier:

$$\mathbf{y}_\ell = \mathrm{sg}[\mathbf{x} - \hat{\mathbf{x}}_\ell], \qquad \mathbf{u}_\ell = \frac{\mathbf{y}_\ell}{\mathrm{L2}(\mathbf{y}_\ell) + \epsilon} \oplus [1], \qquad g_\ell := \mathrm{sg}[\mathrm{L2}(\mathbf{y}_\ell)]. \tag{9}$$

At layer 0 the empty sum gives $\mathbf{r}_0 = \mathbf{0}$ and $\mathbf{y}_0 = \mathbf{x}$, so the first layer classifies the input directly. When gain scaling (`resid_gain`) is enabled, each layer classifies the direction of the current residual and scales its write by $g_\ell$; otherwise $g_\ell = 1$.

**Direct atoms and fibers.** We also evaluate two architectural variants of the dictionary. In the *direct-atom* variant, the shared decoder is omitted and signed atoms live directly in the $d$-dimensional output space, so each atom is an explicit vector in activation space. In the *fiber* variant, each entry carries a local rank-$r$ basis and each head reads $r$ continuous coordinates from its input, augmenting the discrete entry choice with a low-rank continuous correction; training fibers on top of a frozen hard code refines reconstruction without altering the discrete assignments.

**Noise and dropout.** Classifier logits take Gaussian noise $\varepsilon_K$, scaled relative to the logit standard deviation under hard selection; dictionary features take Gaussian noise $\varepsilon_F$ proportional to each coordinate's batch standard deviation, with the scale stop-gradiented so the model cannot suppress noise by shrinking feature variance. Winner dropout sets a head's argmax logit to $-\infty$ with probability $p_{\text{drop}}$.

## 5.2 Training

**Embedding model.** We embed texts using SONAR [22], a multimodal and language-agnostic sentence embedding model consisting of a translation-trained encoder (mean-pooled into a 1024-dimensional vector) and a paired decoder that generates text from that vector in a chosen target language. Because translations map to nearby vectors, reconstructed or steered embeddings can be decoded into a fixed language and re-encoded to measure text fidelity and cycle-consistent steering. Large Concept Models [36] generate sentence embeddings directly in SONAR space, making SONAR a natural testbed for embedding-level interventions, though SONAR decoding is known to be sensitive to small perturbations [36]. As a second substrate where sparse dictionary learning is well established and reconstructions can be spliced directly into a forward pass, we also train models on the residual stream of GPT-2 small [51].

**Loss function.** Reconstruction is trained with a whitened mean-squared error, where $\mathbf{w}$ is the per-dimension inverse variance normalized to mean one,

$$\mathrm{MSE}(X, \hat{X}) := \frac{1}{bd} \sum_{n=1}^{b} \sum_{q=1}^{d} w_q \left(\hat{x}_{nq} - x_{nq}\right)^2, \tag{10}$$

applied with deep supervision [37] at every prefix $\hat{X}_\ell$ ($\ell = 1, \ldots, l$) and averaged across layers, so each layer is trained to explain the residual left by earlier layers. The auxiliary statistics are weighted by $\mathbf{s}$ and summed across layers:

$$L := \frac{1}{l} \sum_{\ell=1}^{l} \mathrm{MSE}(X, \hat{X}_\ell) + \sum_{\ell=0}^{l-1} \mathbf{s} \cdot \boldsymbol{\psi}_\ell, \qquad \boldsymbol{\psi}_\ell := \left[\, \mathrm{L1} \;\; \mathrm{cossim}_b \;\; \mathrm{KL}_m \;\; \omega \,\right]_\ell^\top, \tag{11}$$

where L1 is the batch mean of the feature L1 norm. Because minimizing $\mathrm{cossim}_b$ also reduces per-sample entropy $H$, we set the weight on $H$ to zero. Most models reported below were trained with the predecessor of $\omega$ (the within-sample mean cosine between heads' feature vectors); Section 6.2 reports a direct sweep over the $\omega$ penalty on GPT-2.

### 5.3 Evaluation

We evaluate Ontologizer models against `TopK` SAEs [24] trained on the same cached embeddings under the same whitened MSE objective, reporting whitened fraction of variance unexplained ($FVU_w$) on a held-out evaluation tail. A structural-ablation ladder of SAE variants introduces Ontologizer components one step at a time; two rungs that add per-head competition over a shared decoder are equivalent to single-layer Ontologizers with linear classifiers (`g160top1` with hard selection and `g160softmax` with softmax) and are treated as simplified model variants rather than external baselines. Our evaluation suite measures:

- reconstruction error against code capacity and a Gaussian rate–distortion baseline [15];
- within-head semantic coherence of decoded atoms against size-matched random subsets from the same dictionary;
- steering efficacy and collateral in embedding space (how often a step along a target direction realizes the target entry and how many other heads it flips, relative to a random step of equal norm), along with additivity of two-head interventions;
- cross-seed reproducibility of head partitions and output contributions; and
- blinded autointerp detection.

Because AxBench [56] found that prompting, fine-tuning, and supervised difference-of-means vectors outperform SAE latents at steering while linear probes do not, we compare our unsupervised steering primitives directly against both difference-of-means and linear-probe vectors.

## 6 Preliminary results

Our evaluation is ongoing and the results so far are mixed. This section summarizes the headline result for each line of work; full experimental protocols, ablations, and secondary analyses are reported in the working notes[2].

Three model families appear throughout the SONAR comparisons:

- the softmax Ontologizer `resid_nc`, which stacks $l$=5 layers of $h$=32 heads with $k$=32 entries (a soft code);
- the straight-through Ontologizer `ste_h76_init01`, which stacks $5 \times 76$ heads at $k$=32 (a 1900-bit hard code); and
- `TopK` SAEs trained on the same embedding cache under the same whitened MSE objective.

The SONAR cache contains embeddings from an 86-language mC4 sample. Due to a tagging bug in the original cache, 93% of rows were encoded with SONAR's English source-language tag rather than their own language tag; fixing the tag shifts individual embeddings by roughly twice the hard code's reconstruction error, but changes the share of variance explained by language only slightly (from 7.4% to 6.7%). Reconstruction is scored by $FVU_w$ on the final 32,768 rows of the cache, which are held out from training for all SAEs and straight-through models. Although the softmax Ontologizers were trained before that split was introduced, re-evaluating them on freshly encoded out-of-sample rows changes their $FVU_w$ by at most 0.3%.

[2] K. Wiechecki et al., *Dictionary learning as unsupervised steering-vector generation: Ontologizers vs. sparse autoencoders on SONAR sentence embeddings*, working notes, in preparation; distributed with this proposal.

| point | coeffs | index bits | $FVU_w$ |
|---|---|---|---|
| onto $v$=0 (mean-assignment origin) | 0 | 0 | 1.001 |
| onto hard (argmax) | 0 | 800 | 17.34 |
| *Gaussian reference, 800 bits* | 0 | 800 | *0.226* |
| SAE $k_{SAE}$=32 tier | 32 | 276–313 | 0.494–0.509 |
| onto dev $v$=1 | 160 | 800 | 0.677 |
| SAE $k_{SAE}$=160 | 160 | 1206 | 0.258 |
| onto dev $v$=4 | 640 | 2421 | 0.289 |
| onto dev $v$=8 | 1280 | 3732 | 0.094 |
| SAE $k_{SAE}$=5120 (L0≈1520) | 1520 | 6423 | **0.00011** |
| 1-layer soft variant `g160softmax` | 5120 | 0 | 0.00436 |
| onto soft ($v$=32) | 5120 | 0 | 0.00538 |

**Table 1** Reconstruction against code capacity for the softmax Ontologizer `resid_nc`. Ontologizer points truncate each head to its $v$ largest deviations from the mean assignment; SAE points use measured L0. Index bits specify the active coefficient set: $l\,h\log_2\binom{k}{v}$ for the Ontologizer and $\log_2\binom{m}{\mathrm{L0}}$ for an SAE of $m$ latents. The Gaussian reference is computed by reverse water-filling on the evaluation tail's whitened covariance [15, Thm. 10.3.3] and gives the minimum distortion at that rate for a Gaussian source with the same covariance.

## 6.1 The price of a soft code

Imposing contrast-set structure on a soft code carries a steep reconstruction penalty. Across all capacities tested, standard `TopK` SAEs dominate the softmax Ontologizer's truncated deviation code, which retains the $v$ entries per head furthest from the mean assignment and pins the rest to the mean (Table 1). At $FVU_w \approx 0.49$, an SAE transmits 32 active coefficients whereas the Ontologizer requires 320, and at matched capacity the unstructured SAE reconstructs roughly 50× more accurately. Furthermore, taking the one-hot argmax of the softmax model fails to reconstruct the input at all, yielding $FVU_w = 17.34$. That is 77× higher than the Gaussian rate–distortion bound at 800 bits (0.226), which upper-bounds the optimal distortion achievable on any source with the same covariance [15]. In a soft model, the discrete argmax entries that an intervention forces are therefore not what carries the reconstruction, which motivates training hard codes directly.

## 6.2 Trained atoms form contrast sets

Within each head, the learned atoms exhibit the negative cosine similarity expected of a contrast set. In 96% of the softmax Ontologizer's heads, the decoded atoms have lower pairwise cosine similarity than size-matched random subsets of atoms drawn from the same dictionary (Table 2), and this repulsion is even stronger under the whitened inner product used in training. Thus, a soft head's entries represent separated alternatives rather than a cluster of similar directions. The hard-code stack shows a similar pattern in its weights, with within-head decoded atoms roughly orthogonal to slightly negatively correlated (mean cosine 0.023 in layer 0 and −0.015 to −0.020 in deeper layers). Because each head's assignments sum to one, however, adding a constant vector to one head's atoms and subtracting it from another head in the same layer leaves the layer's output unchanged; only differences between atoms within a head are uniquely determined by the function, and a centered-atom null comparison remains to be run. By contrast, `g160top1`—which applies hard per-head selection in a single layer with unconstrained per-head magnitudes—learns positively correlated atoms within each head.

Between heads, however, the intended coordinate partition does not form in summed models. Across all summed models tested, the support overlap $\omega$ (Eq. (7)) remains at or above a coordinate-shuffled null, with each head's usage spanning 52–98% of dictionary coordinates rather than the ∼3% expected of a 32-

| heads or grouping | count | mean $\zeta_{\cos}$ | frac $\zeta>2$ | frac $\zeta<-2$ |
|---|---|---|---|---|
| `g160top1` (trained, hard) | 160 | +161.3 | 1.00 | 0.00 |
| $k_{\text{SAE}}$=32 SAE, discovered groups | 359 | +1.7 | 0.36 | 0.02 |
| Ontologizer heads (trained, soft) | 160 | −17.6 | 0.04 | **0.96** |
| `g160softmax` (trained, soft) | 160 | −14.7 | 0.00 | **1.00** |

**Table 2** Within-head decode-direction coherence $\zeta_{\cos}$, standardized against size-matched same-pool nulls ($n_{\text{null}}$=500). Soft heads are anti-coherent: their atoms are pushed apart.

way partition. This overlap persists in the subspace visible to the decoder, and directly penalizing $\omega$ on GPT-2 reduces dictionary-space overlap only by pushing 73–95% of atom energy into the decoder's null or near-null space while decoded overlap stays 8–10% above the null (working notes). Summed heads therefore learn within-head contrast sets that share dictionary coordinates, whereas concatenated heads enforce a coordinate partition by construction.

## 6.3 A discrete code beats TopK SAEs up to 160 latents, not every sparse code

With straight-through selection, the forward pass uses the discrete argmax directly. At 1900 index bits, the $5 \times 76$ hard code achieves $\text{FVU}_w = 0.154$ using only four scalar gains beyond its discrete choices, and pinning those gains to their training-set means yields a purely discrete 1900-bit code at $\text{FVU}_w = 0.158$ (Table 3). By comparison, a sparse code that also transmits continuous coefficients must divide its bit budget between support indices and quantized magnitudes, and no quantization of an SAE's coefficients can improve on an unconstrained least-squares refit on its active support. A $k_{\text{SAE}}$=160 TopK SAE spends 1206 bits to index its active set, leaving 4.3 bits per coefficient at a 1900-bit budget, and even its unquantized support refit has $\text{FVU}_w = 0.236$; at a matched 1900-bit budget, the pure hard code therefore reduces error by at least 33%.

However, SAEs with variable sparsity or wider supports can outperform the hard stack under entropy coding. When coefficients are quantized and each latent's symbols are entropy-coded, a weakly penalized ReLU+L1 SAE reaches $\text{FVU}_w = 0.129$ at an average rate of 1900 bits per row, because skewed latent firing frequencies reduce index entropy and allow variable-length codes across rows (whereas the hard code uses entries nearly uniformly at a fixed 1900 bits per row). When every row is capped at 1900 bits, the L1 SAE reaches 0.152 (comparable to the hard code's 0.158), and under fixed-length support indexing the hard code leads (0.158 vs. 0.180). On the sentence-split SONAR cache, a wider TopK SAE ($k_{\text{SAE}}$=256 active out of 11,264 latents) beats the hard stacks under both average-rate and per-row budgets (0.128 and 0.129 vs. 0.161 and 0.164), trailing only when restricted to a single global quantization step without per-row bit allocation (0.185 vs. 0.164). Relative to the Gaussian rate–distortion bound at 1900 bits, the hard stack's distortion remains 3.1× higher, reflecting the gap between single-sample quantization and high-dimensional block coding [15].

## 6.4 Depth helps reconstruction and costs addressability

Comparing a five-layer stack of 76 heads per layer against a single layer of 380 heads isolates the effect of residual depth at matched bit budget (1900 bits), parameter count, and near-uniform entry usage. The five-layer stack achieves 1.79× lower reconstruction error ($\text{FVU}_w$ 0.1535 vs. 0.2750). Whereas a single layer of heads acts like a parallel additive quantizer [3] in which heads select atoms independently without observing one another's residuals (which is benign only when heads occupy orthogonal subspaces as in product quantization [30]), the residual stack conditions each layer on the error left by previous layers, as in multi-stage residual quantization [58], while also applying deep supervision to

| code | coeffs | index bits | $\mathrm{FVU}_w$ | at 1900 bits | every row |
|---|---|---|---|---|---|
| **straight-through Ontologizer,** $5 \times 76$ | **4 gains** | **1900** | **0.154** | **0.155** | — |
| gains pinned to their means | 0 | 1900 | 0.158 | — | 0.158 |
| SAE, `TopK`, $k_{\mathrm{SAE}}$=160 | 160 | 1206 | 0.258 (0.236) | 0.259 | 0.258 |
| SAE, ReLU+L1 $3 \times 10^{-5}$ | 519 | 2418 | 0.219 | 0.242 | 0.250 |
| SAE, ReLU+L1 $10^{-5}$ | 1438 | 4380 | 0.048 | 0.175 | 0.200 |
| SAE, ReLU+L1 $3 \times 10^{-6}$ | 1637 | 4623 | 0.005 | **0.129** | **0.152** |
| 1-layer hard variant `g160top1` | 160 | 800 | 0.274 (0.273) | 0.274 | 0.274 |

**Table 3** Comparison of the hard code against converged SAEs on the held-out tail. Values in parentheses show the least-squares refit on each code's active support, which lower-bounds the error achievable by any quantization of its coefficients. "At 1900 bits" reports each code's error at an average rate of 1900 bits per sample with quantized values and per-slot entropy coding, while "every row" caps each row at 1900 bits (matching the fixed-length pinned hard code). `g160top1` uses hard per-head selection in a single layer of linear classifiers and retains a scalar magnitude per winning entry.

each prefix.

However, residual depth makes individual heads harder to target via input-space perturbations. Stepping in embedding space by $0.25\|\mathbf{x}\|$ along a target entry's decoded atom flips the target head to that entry on 99.6% of rows in the single-layer model while flipping 33% of other heads, whereas in the five-layer stack it realizes the target entry on 37% of rows while flipping 66% of other heads. In both models, stepping along a decoded atom flips as many unrelated heads as a random step of the same norm, both because the hard classifiers sit close to their decision boundaries (median logit margin $1.5 \times 10^{-4}$ between winner and runner-up) and because changes to an early layer alter the residual seen by all subsequent layers. Among our hard-selection models, only the single-layer linear variant `g160top1` achieves selective input-space addressing, realizing 58% of targets while flipping 2.6% of other heads (vs. 23% for a random step).

## 6.5 Steering

Evaluated directly in embedding space, soft-model steering reliably realizes target entries: stepping by $0.25\|\mathbf{x}\|$ along a target entry's decoded atom switches the head's argmax to that entry on 82% of rows, compared to 2% for a random step of equal norm (an effect previously obscured by text round-tripping, since decoding and re-encoding an unsteered embedding preserves a head's argmax on only 11–13% of rows). When interventions are applied directly to head assignments, two-head edits compose nearly additively in decode space, with a median cosine of at least 0.95 between the joint decode change and the sum of the individual changes across all layers (and exact equality in the final layer, where no downstream layers reclassify the residual). In `resid_nc`, forcing a single layer-0 entry consistently translates decoded text into Burmese while preserving semantic content, providing a concrete existence proof of a causal steering entry on one seed.

When we compare decoded-atom steps against supervised vectors fitted to the model's own entry partitions at step size $0.25\|\mathbf{x}\|$, the softmax stack's decoded atoms reach their target entry on 75% of rows (vs. 46–47% for difference-of-means and logistic-probe vectors), whereas the hard stack's decoded atoms reach 37% (vs. 91–94% for the supervised vectors). However, two arbitrary-partition baselines show that high target realization in embedding space is a generic property of partitioning a vector space rather than evidence that the learned partitions capture meaningful factors. In an untrained $5 \times 76$ stack, parametric decoded-atom steps realize targets at chance (2%), but supervised difference-of-means and probe vectors reach 75% and 94% (and classifier gradients reach 61% and 85%, vs. 30% and 16% in the trained stack) with one third the collateral flips of the trained stack (22% vs. 66%). Similarly,

| arm | eff (native) | eff (dm frame) | hit rate | collateral |
|---|---|---|---|---|
| difference of means | 0.474 | 0.474 | 0.062 | 0.752 |
| entry forcing | 0.399 | 0.302 | 0.000 | 0.776 |
| linear probe | 0.294 | 0.072 | 0.000 | 0.701 |
| random direction | – | – | – | 0.705 |

**Table 4** Language steering under matched intervention budgets on the softmax Ontologizer `resid_nc_hm`; medians over 24 language–magnitude conditions per arm. Effect is displacement along the arm's own direction (native) and along the shared difference-of-means direction (dm frame); hit rate is the fraction of steered decodes reaching the median projection of genuine target-language rows; collateral is content change elsewhere, with the random direction as its floor.

in a single layer of 380 random-projection heads ($k$=32), empirical cell-centroid swaps realize 78% of targets (67% at native step size) and projection directions, margin gradients, and logistic probes reach 87–100% at 30–32% collateral, matching the trained single-layer hard model. On an external ground-truth attribute—language steering under matched intervention budgets (Table 4)—forcing a soft-model entry moves decoded text 0.64× as far along the supervised difference-of-means axis as supervised steering itself and 4× as far as a linear probe, though all methods (including supervised steering) incur collateral content changes comparable to a random perturbation of the same norm.

## 6.6 The error reproduces; the heads do not

Two hard-code runs initialized with different random seeds converge to the same held-out error ($\mathrm{FVU}_w$ = 0.1535 on both seeds), but their individual heads do not match: Hungarian-matched head partitions have a mean NMI of 0.012 (against a 0.005 permutation null) and matched head output contributions have a centered cosine similarity of 0.007 (null 0.002), whereas comparing a single run against an earlier checkpoint 19,500 steps prior yields 0.38 partition NMI and 0.98–0.99 atom cosine similarity. Even the overall reconstructions diverge across seeds: the two hard-code seeds' whitened residuals correlate at 0.30, compared to 0.83 between two seeds of a $k_{\mathrm{SAE}}$=32 SAE. Thus, heads stabilize within a training run but converge to different decompositions across seeds. The same pattern holds on GPT-2, where only layer-0 heads show above-chance cross-seed agreement (at less than a fifth of the within-run control), and in the softmax Ontologizer, where at most 0.1% of entries match across seeds compared to 3–31% for a $k_{\mathrm{SAE}}$=32 SAE pair.

Because layer 0 is the only layer whose input is not conditioned on earlier layers' discrete choices, it is the only layer where individual heads recur without additional supervision: in every SONAR run, the largest-norm layer-0 head partitions inputs by broad topic and document genre (such as crime news, geopolitics, encyclopedic articles, manufacturing, marketing, legislation, and entertainment, with blinded contrastive descriptions detected at F1 0.52 vs. a 0.27 null). By contrast, source language—the primary labeled attribute in the corpus—is distributed across multi-head neighborhoods rather than isolated in a single head, as expected for a language-agnostic embedding space. Achieving cross-seed reproducibility across a wider set of heads requires an explicit paired-view training signal (Section 6.7).

## 6.7 Paired sentences make a block of heads reproduce

To test whether temporal or document-level consistency can break the symmetry across seeds, we trained a single-layer model of 380 heads ($k$=32) on pairs of sentences drawn from the same document in a 44.1M-segment sentence-split SONAR cache (`mc4_sent16`), adding a Fano lower bound on the mutual information between the entries assigned to the two sentences for the first 38 "slow" heads. After three epochs, these 38 slow heads maintain the same entry across adjacent sentences at Cohen's kappa

| model | code per token | FVU | loss recovered |
|---|---|---|---|
| hard code, direct atoms (10M tokens) | 800 bits + 5 gains | 0.217 | 97.0% |
| **hard code, direct atoms (40M tokens)** | 800 bits + 5 gains | 0.195 | **97.6%** |
| hard code, concatenated heads | 800 bits + 5 gains | 0.186 | 97.5% |
| hard code, concatenated heads, $k$=64 | 960 bits + 5 gains | **0.164** | **97.9%** |
| TopK SAE, $k_{\text{SAE}}$=32 | 32 coeffs + ~276 bits | 0.189 | 96.2% |
| TopK SAE, $k_{\text{SAE}}$=128 | 128 coeffs + ~859 bits | 0.116 | 98.6% |

**Table 5** GPT-2 small, residual stream entering block 8, held-out documents. Loss recovered is $(\text{CE}_{\text{ablate}} - \text{CE}_{\text{recon}})/(\text{CE}_{\text{ablate}} - \text{CE}_{\text{clean}})$ with the residual mean-ablated as the floor; FVU is unwhitened here. Concatenated heads write disjoint 48-dimensional blocks through a shared decoder. SAEs train 10 epochs over the 10M-token cache (556–588 total bits for $k_{\text{SAE}}$=32 at ~5 bits per coefficient). One seed per Ontologizer (10M direct seed-43 replica: FVU 0.219, 96.9%).

0.46, compared to 0.07 in an unregularized control. Across three random seeds, the slow blocks learn matching partitions: one-to-one Hungarian matching yields a mean NMI of 0.412–0.414 for every pair of seeds (against a 0.005 null), every slow head in one seed pairs with a slow head in the other (chance 10%), and within-language agreement across 84 languages remains at NMI 0.312–0.314 (vs. 0.041 null), whereas the remaining 342 unregularized heads and the control model's heads match at only 0.016–0.017. Moreover, the top 38 slow directions extracted from the one-hot slow-head code achieve a higher held-out cross-sentence correlation than the top 38 linear slow-feature directions of the raw embedding (0.72–0.73 vs. 0.636 overall, and 0.52 vs. 0.46 after demeaning each language). The agreement holds for the block rather than for individual heads: two slow heads within one seed already agree at NMI 0.40, any one-to-one pairing of two seeds' slow heads scores 0.397–0.399, and the three seeds' pairwise matchings compose for only 6 of the 38 heads (1 expected by chance), so no individual slow head is yet identified across runs. An ordered penalty on dependence between the slow heads' partitions does not repair this without loss: at a small weight no head is identified, and at a large weight exactly one is (over two seeds, NMI 0.386 with its counterpart against 0.009 with any other slow head) while the other 37 lose their slow content, and the block then explains 0.43–0.44 of the variance of the eight slowest linear directions where the Fano block explains 0.76–0.77 and a single slow head 0.38–0.43. The result is further limited to a one-layer model and costs 1.8–1.9% in $\text{FVU}_w$, and the redundant slow heads align primarily with the top one or two linear slow directions, leaving open how many distinct document-level variables the block captures.

## 6.8 Hard codes on GPT-2

On GPT-2 small layer-8 residuals, a five-layer 800-bit hard code with signed direct atoms and no decoder recovers a larger fraction of the model's next-token cross-entropy loss when spliced into the forward pass than a $k_{\text{SAE}}$=32 TopK SAE, both on the 10M-token training cache (97.0% vs. 96.2% loss recovered, despite a higher unwhitened FVU of 0.217 vs. 0.189) and after one pass over 40M tokens (97.6% at FVU 0.195; Table 5). When the SAE's 32 active coefficients are quantized to ~5 bits each (556–588 total bits vs. 800 + ~25 for the hard code), however, the SAE achieves lower FVU per bit; across model families, raw FVU does not monotonically predict downstream loss recovery. Replacing the shared decoder in the 40M-token concatenated-head model with separate per-layer decoders reduces FVU from 0.186 to 0.169 (97.3% → 97.7% loss recovered) and reduces cross-layer coupling between co-indexed heads from 97% to 6% (chance 3%, though a width-matched shared-decoder control has not yet been run). Training continuous low-rank fibers on top of the frozen discrete code recovers 99.9% of the loss without changing the discrete assignments, at the cost of 640 continuous values per token.

Describability and steering diagnostics on these GPT-2 codes highlight two important limitations.

First, blinded LLM-judge balanced accuracy for entry descriptions (chance 0.50; document-split nulls 0.496–0.507) falls off rapidly after the first layer: layer 0 averages 0.70–0.71 (0.91 on highest-margin examples, 0.65 on random members, and 0.55 near the boundary), layer 1 drops to 0.56–0.57, and layers 2–4 sit at chance (0.50–0.52). Moreover, in layer 0 a lookup baseline that uses only the current token's identity outperforms the LLM judge on single entries (0.75 vs. 0.69 on concatenated heads and 0.74 vs. 0.71 on direct atoms), and the +0.10 accuracy gain from conditioning on two-head pairs in the concatenated model comes largely from lexical selectivity (+0.01 on direct atoms). Second, while single-head edits perturb next-token predictions modestly (setting a layer-0 head to its mean entry costs 0.02–0.075 nats vs. 0.055 nats for zeroing an active SAE latent, and entry swaps cost 0.001–0.06 nats) and swapping a head to entry $j'$ shifts next-token probabilities toward $j'$'s empirical distribution (mean target rank 3.5–6 out of 31 at unit step and 5–8 at 4× step, vs. 14–18 for random directions), centroid steps between random-projection bins achieve equally specific next-token steering across all five layers (mean rank 0.21–1.69 vs. 0.56–1.37 for learned entries) because the downstream map to the logits is locally nearly linear.

## 6.9 Automated interpretability

Standard per-latent autointerp metrics evaluate individual directions in isolation rather than contrastive partitions, and across six models blinded detection F1 tracks log activation density (correlation −0.89) rather than architecture, with the Ontologizer lying on the same density trend as the SAEs. Much of the apparent advantage of sparse SAEs on the multilingual cache comes from single-language features: latents whose top activating examples come from a single language can be described simply by naming that language, achieving detection F1 of 0.90–0.97 against random negative examples, and restricting the comparison to features with corpus-typical language diversity narrows the sparse SAE's F1 lead over the Ontologizer from 0.17 to 0.10. Finally, generating descriptions by decoding atom directions from parameters alone without activating examples fails across all models (F1 0.04 for the Ontologizer and at most 0.14 for any model, below the 0.67 always-positive baseline), confirming that a single decoded atom vector does not suffice to name a category without activation examples.

## 6.10 Against the desiderata

We can summarize where the architecture stands relative to the three desiderata in the introduction:

- **Causal:** Qualified positive. Target entries can be realized by embedding-space steps and used to steer decoded outputs, but stepping in input space flips unrelated heads as often as a random step of the same norm (compounded by the residual stack), and on both SONAR and GPT-2 swaps between arbitrary random-projection or untrained cells steer toward their own empirical distributions just as specifically as learned entries.
- **Disentangled:** Positive for direct code-level interventions, where changing a head's assignment modifies only that head and composes additively with edits to other heads.
- **Closed-form:** Negative so far. Parameter-only decodes fail to produce valid feature descriptions, and summed models do not partition dictionary coordinates across heads without explicit concatenation.

Underlying all three desiderata is cross-seed identifiability, which the original proposal took for granted: past the first layer, independent training runs learn different heads, and only a block of heads trained with a paired-sentence objective has reproduced across seeds so far, as a block rather than head by head (Section 6.7). On reconstruction fidelity, however, moving from soft to straight-through hard selection closes the gap with moderate-width TopK SAEs: the 1900-bit hard code outperforms TopK SAEs up to 160

active latents at matched total bits on SONAR (though not wider or entropy-coded variable-rate SAEs) and recovers more next-token loss than a $k_{\text{SAE}}$=32 SAE on GPT-2.

# 7 Future work

## 7.1 Identifiability first

Before individual heads can be interpreted or used in audits, we need to ensure that they reproduce across training runs, or else identify the subset of heads that do. In our current models only the first layer shows partial reproducibility, with a single topic-and-genre head recurring across SONAR runs; understanding why that head recurs and how to extend reproducibility to other heads is our primary focus. Classical identifiability theory points to two approaches:

1. selecting reproducible heads or runs via unsupervised cross-seed agreement [20] (in latent-variable models, three or more conditionally independent views of a discrete variable identify its distribution up to permutation when the class-conditional distributions satisfy a rank condition [2], with individual heads serving as candidate views here); and

2. breaking the symmetry of the marginal distribution [38] using paired observations such as adjacent sentences in a document [28, 34, 39, 32, 60], since standard sparse-coding uniqueness guarantees require lossless reconstruction [26].

Initial experiments on the 44.1M-segment sentence-split SONAR cache (`mc4_sent16`; working notes) show that a sentence embedding $\mathbf{x}_t$ linearly explains 10.4% of the variance of the following sentence $\mathbf{x}_{t+1}$ ($\text{FVU}_{\text{next}}$=0.896), and that adjacent sentences are strongly aligned in direction (raw cosine AUC 0.923 vs. 0.685 in whitened distance), yet heads from single-sentence models switch entries on 92.2–98.1% of adjacent pairs. Directly penalizing entry switches across adjacent sentences on 38 designated slow heads collapses their usage entropy to 0.04–0.09 bits for penalty weights $s_{\text{pair}} \geq 10^{-4}$. Replacing that penalty with a Fano lower bound on each slow head's mutual information across the sentence pair prevents collapse and produces a block of slow heads that reproduces across three seeds, though not head by head (Section 6.7). A first attempt to tell the slow heads apart, an ordered penalty on dependence between their partitions, identifies one head at full weight and strips the rest of their slow content. Immediate next steps include: separating the slow heads, which agree with one another nearly as well as with another seed's, by a criterion that keeps what the block carries jointly; extending the objective to sentence triples to test the three-view conditional-independence rank conditions on held-out data [2]; characterizing what the slow heads encode beyond language; and applying the paired-view objective to a causal language model over multi-turn conversations (such as Gemma 3 1B instruct). In parallel, replacing the shared decoder in the 40M-token GPT-2 concatenated-head model with per-layer decoders reduces cross-layer same-head coupling from 97% to 6% while improving FVU from 0.186 to 0.169 (pending a width-matched shared-decoder control).

## 7.2 Steering at scale

We have compared model-derived steering directions against supervised vectors on the models' own partitions (working notes) and, for the softmax model, on language steering (Section 6.5). Several steering evaluations remain open: running the language-steering comparison on hard codes; evaluating AxBench-style steering toward external concepts against supervised vectors and SAE latents at tuned intervention strengths (initial Gemma 3 1B and Gemma 4 A4B models with per-layer decoders are currently training); testing bilinear eigenfeatures as steering directions; measuring whether an entry

swap produces consistent semantic shifts across contexts; and trading off the single-layer model's 1.79× higher reconstruction error against its near-perfect target realization and lower collateral.

Reducing input-space collateral requires widening classifier margins rather than merely finding better steering directions: in the hard stack, heads sit near their decision boundaries, and even oracle supervised vectors reduce collateral flips by at most 0.1 relative to a random step of the same norm. Reducing the number of entries per head increases those margins. In a five-layer stack of binary ($k$=2) heads, a step of norm $0.25\|\mathbf{x}\|$ is predicted to flip 13% of same-layer heads to first order (vs. 30% for $k$=32), and empirically flips 17–19% of other heads across all directions (vs. 66% for the $k$=32 stack), matching that lower collateral at the 32-entry stack's step count under a matched per-head $\mathrm{KL}_m$ weight. Because that binary run used a single seed and half the batch size, however, a controlled comparison isolating $k$ is still needed, with the single-layer linear model `g160top1` (Section 6.4) serving as the baseline for low-collateral addressing.

## 7.3 Task reconstruction

For heads that reproduce across seeds, we aim to identify the classification task each head performs by measuring mutual information with metadata labels (such as topic and genre), generating contrastive natural-language descriptions that distinguish an entry from its sibling entries (scored by blinded $k$-way assignment accuracy against a $1/k$ chance baseline), and using parallel translations to separate content-sensitive heads from surface-form heads.

## 7.4 Disjoint heads

Because summed models fail to partition dictionary coordinates across heads and penalizing $\omega$ on dictionary weights merely pushes atom energy into the decoder's near-null space (leaving decoded overlap 8–10% above the coordinate-shuffle null; working notes; cf. [38, §5.1]), any learned overlap penalty must operate on decoded or direct atoms where the decoder cannot absorb it. Alternatively, concatenated heads enforce disjoint dictionary blocks by construction and already outperform direct atoms on GPT-2 when each head's coordinate slice is wider than $k$ (Section 6.8).

## 7.5 Transcoder adaptation

To adapt the architecture into a transcoder that predicts a sublayer's output $Y$ from its input $X$ [21], we tested two pilot designs on GPT-2 block 8's MLP (10M tokens, concatenated heads; working notes). A dual-decoder stack that reconstructs both $X$ and $Y$ from the shared residual accumulator beats the linear ridge baseline on $Y$ ($\mathrm{FVU}_Y$ 0.769, 28.3% MLP-splice loss recovered; ridge on SAE latents reaches 0.612) only when the loss weight on $Y$ is raised to $\lambda \geq 3$ ($\mathrm{FVU}_Y$ 0.763 → 0.729, 28.6% → 32.1% loss recovered), which severely degrades input reconstruction ($\mathrm{FVU}_X$ 0.284 → 0.475). Conversely, a target-free forwarding mode (`forward="cond"`) that does not require $Y$ at inference time reaches only $\mathrm{FVU}_Y = 0.855$ (10.0% loss recovered), with deeper layers increasing error (0.823 → 0.855). A viable multi-layer transcoder will require a residual recurrence that iteratively refines predictions of $Y$ without access to the ground-truth target during the forward pass.

# Acknowledgements

Design, typesetting, and editorial production of this preprint are by The Flock Capital Research Department.

## References


[1] Eirikur Agustsson et al. “Soft-to-Hard Vector Quantization for End-to-End Learning Compressible Representations”. In: *Advances in Neural Information Processing Systems*. Vol. 30. 2017. arXiv: 1704.00648 [cs.LG]. URL: https://arxiv.org/abs/1704.00648.

[2] Elizabeth S. Allman, Catherine Matias, and John A. Rhodes. “Identifiability of Parameters in Latent Structure Models with Many Observed Variables”. In: *The Annals of Statistics* 37.6A (2009), pp. 3099–3132. DOI: 10.1214/09-AOS689. arXiv: 0809.5032 [math.ST]. URL: https://arxiv.org/abs/0809.5032.

[3] Artem Babenko and Victor Lempitsky. “Additive Quantization for Extreme Vector Compression”. In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*. 2014, pp. 931–938. URL: https://openaccess.thecvf.com/content_cvpr_2014/html/Babenko_Additive_Quantization_for_2014_CVPR_paper.html.

[4] Christopher F. Barnes, Syed A. Rizvi, and Nasser M. Nasrabadi. “Advances in Residual Vector Quantization: A Review”. In: *IEEE Transactions on Image Processing* 5.2 (1996), pp. 226–262. DOI: 10.1109/83.480761.

[5] Yoshua Bengio, Nicholas Léonard, and Aaron Courville. *Estimating or Propagating Gradients Through Stochastic Neurons for Conditional Computation*. 2013. arXiv: 1308.3432 [cs.LG]. URL: https://arxiv.org/abs/1308.3432.

[6] Nick Bostrom. *Superintelligence: Paths, Dangers, Strategies*. Oxford: Oxford University Press, 2014. ISBN: 978-0-19-967811-2.

[7] Samuel R. Bowman et al. *Measuring Progress on Scalable Oversight for Large Language Models*. 2022. arXiv: 2211.03540 [cs.HC]. URL: https://arxiv.org/abs/2211.03540.

[8] Trenton Bricken et al. “Towards Monosemanticity: Decomposing Language Models With Dictionary Learning”. In: *Transformer Circuits Thread* (2023). URL: https://transformer-circuits.pub/2023/monosemantic-features/index.html.

[9] Collin Burns et al. “Discovering Latent Knowledge in Language Models Without Supervision”. In: *International Conference on Learning Representations*. 2023. URL: https://arxiv.org/abs/2212.03827.

[10] Bart Bussmann et al. “Learning Multi-Level Features with Matryoshka Sparse Autoencoders”. In: *Proceedings of the 42nd International Conference on Machine Learning*. Vol. 267. Proceedings of Machine Learning Research. PMLR, 2025, pp. 6077–6101. arXiv: 2503.17547 [cs.LG]. URL: https://arxiv.org/abs/2503.17547.

[11] Tue M. Cao et al. *Tree SAE: Learning Hierarchical Feature Structures in Sparse Autoencoders*. 2026. arXiv: 2605.07922 [cs.LG]. URL: https://arxiv.org/abs/2605.07922.

[12] Lawrence Chan, Leon Lang, and Erik Jenner. *Natural Abstractions: Key Claims, Theorems, and Critiques*. 2023. URL: https://www.lesswrong.com/posts/gvzW46Z3BsaZsLc25/natural-abstractions-key-claims-theorems-and-critiques-1.

[13] David Chanin et al. “A is for Absorption: Studying Feature Splitting and Absorption in Sparse Autoencoders”. In: *Advances in Neural Information Processing Systems*. Vol. 38. 2025, pp. 91489–91526. arXiv: 2409.14507 [cs.CL]. URL: https://arxiv.org/abs/2409.14507.

[14] Bilal Chughtai, Lawrence Chan, and Neel Nanda. “A Toy Model of Universality: Reverse Engineering How Networks Learn Group Operations”. In: *Proceedings of the 40th International Conference on Machine Learning*. Vol. 202. Proceedings of Machine Learning Research. PMLR, 2023, pp. 6243–6267. URL: https://proceedings.mlr.press/v202/chughtai23a.html.

[15] Thomas M. Cover and Joy A. Thomas. *Elements of Information Theory*. 2nd ed. Wiley, 2006.

[16] James P. Crutchfield. “Way More than the Sum of Their Parts: From Statistical to Structural Mixtures”. In: *Entropy* 28.1 (2026), p. 111. DOI: 10.3390/e28010111.

[17] Hoagy Cunningham et al. *Sparse Autoencoders Find Highly Interpretable Features in Language Models*. 2023. arXiv: 2309.08600 [cs.LG]. URL: https://arxiv.org/abs/2309.08600.

[18] Will Dorrell et al. “Range, not Independence, Drives Modularity in Biologically Inspired Representations”. In: *International Conference on Learning Representations*. 2025. arXiv: 2410.06232 [q-bio.NC]. URL: https://arxiv.org/abs/2410.06232.

[19] Evan Duan. *Pre-Intervention Prediction of Sparse Autoencoder Steering Side Effects*. 2026. arXiv: 2606.08365 [cs.LG]. URL: https://arxiv.org/abs/2606.08365.

[20] Sunny Duan et al. “Unsupervised Model Selection for Variational Disentangled Representation Learning”. In: *International Conference on Learning Representations*. 2020. arXiv: 1905.12614 [cs.LG]. URL: https://arxiv.org/abs/1905.12614.

[21] Jacob Dunefsky, Philippe Chlenski, and Neel Nanda. “Transcoders Find Interpretable LLM Feature Circuits”. In: *Advances in Neural Information Processing Systems*. Vol. 37. 2024. arXiv: 2406.11944. URL: https://arxiv.org/abs/2406.11944.

[22] Paul-Ambroise Duquenne, Holger Schwenk, and Benoît Sagot. *SONAR: Sentence-Level Multimodal and Language-Agnostic Representations*. 2023. arXiv: 2308.11466 [cs.CL]. URL: https://arxiv.org/abs/2308.11466.

[23] Thomas Fel et al. “Into the Rabbit Hull: From Task-Relevant Concepts in DINO to Minkowski Geometry”. In: *International Conference on Learning Representations*. 2026. arXiv: 2510.08638 [cs.CV]. URL: https://arxiv.org/abs/2510.08638.

[24] Leo Gao et al. *Scaling and evaluating sparse autoencoders*. 2024. arXiv: 2406.04093 [cs.LG]. URL: https://arxiv.org/abs/2406.04093.

[25] Ryan Greenblatt et al. *Alignment Faking in Large Language Models*. 2024. arXiv: 2412.14093 [cs.AI]. URL: https://arxiv.org/abs/2412.14093.

[26] Christopher J. Hillar and Friedrich T. Sommer. *When Can Dictionary Learning Uniquely Recover Sparse Data from Subsamples?* 2015. arXiv: 1106.3616 [q-bio.NC]. URL: https://arxiv.org/abs/1106.3616.

[27] Jesse Hoogland et al. “Loss Landscape Degeneracy and Stagewise Development in Transformers”. In: *Transactions on Machine Learning Research* (2025). arXiv: 2402.02364 [cs.LG]. URL: https://openreview.net/forum?id=45qJyBG8Oj.

[28] Aapo Hyvärinen and Hiroshi Morioka. “Nonlinear ICA of Temporally Dependent Stationary Sources”. In: *Proceedings of the 20th International Conference on Artificial Intelligence and Statistics*. Vol. 54. Proceedings of Machine Learning Research. PMLR, 2017. URL: https://proceedings.mlr.press/v54/hyvarinen17a.html.

[29] Eric Jang, Shixiang Gu, and Ben Poole. “Categorical Reparameterization with Gumbel-Softmax”. In: *International Conference on Learning Representations*. 2017. arXiv: 1611.01144. URL: https://arxiv.org/abs/1611.01144.

[30] Hervé Jégou, Matthijs Douze, and Cordelia Schmid. “Product Quantization for Nearest Neighbor Search”. In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 33.1 (2011), pp. 117–128.

[31] Biing-Hwang Juang and A. H. Gray Jr. “Multiple Stage Vector Quantization for Speech Coding”. In: *ICASSP ’82. IEEE International Conference on Acoustics, Speech, and Signal Processing*. Vol. 7. 1982, pp. 597–600. DOI: 10.1109/ICASSP.1982.1171604.

[32] Ilyes Khemakhem et al. “Variational Autoencoders and Nonlinear ICA: A Unifying Framework”. In: *Proceedings of the 23rd International Conference on Artificial Intelligence and Statistics*. Vol. 108. Proceedings of Machine Learning Research. PMLR, 2020. arXiv: 1907.04809 [stat.ML]. URL: https://arxiv.org/abs/1907.04809.

[33] Jin-Hwa Kim et al. “Hadamard Product for Low-rank Bilinear Pooling”. In: *International Conference on Learning Representations*. 2017.

[34] David Klindt et al. “Towards Nonlinear Disentanglement in Natural Data with Temporal Sparse Coding”. In: *International Conference on Learning Representations*. 2021. arXiv: 2007.10930 [stat.ML]. URL: https://arxiv.org/abs/2007.10930.

[35] Anton Korznikov et al. *OrtSAE: Orthogonal Sparse Autoencoders Uncover Atomic Features*. 2025. arXiv: 2509.22033 [cs.LG]. URL: https://arxiv.org/abs/2509.22033.

[36] LCM team et al. *Large Concept Models: Language Modeling in a Sentence Representation Space*. 2024. arXiv: 2412.08821 [cs.CL]. URL: https://arxiv.org/abs/2412.08821.

[37] Chen-Yu Lee et al. "Deeply-Supervised Nets". In: *Proceedings of the Eighteenth International Conference on Artificial Intelligence and Statistics*. Vol. 38. Proceedings of Machine Learning Research. PMLR, 2015, pp. 562–570. URL: https://proceedings.mlr.press/v38/lee15a.html.

[38] Francesco Locatello et al. "Challenging Common Assumptions in the Unsupervised Learning of Disentangled Representations". In: *Proceedings of the 36th International Conference on Machine Learning*. Vol. 97. Proceedings of Machine Learning Research. PMLR, 2019, pp. 4114–4124. arXiv: 1811.12359 [cs.LG]. URL: https://proceedings.mlr.press/v97/locatello19a.html.

[39] Francesco Locatello et al. "Weakly-Supervised Disentanglement Without Compromises". In: *Proceedings of the 37th International Conference on Machine Learning*. Vol. 119. Proceedings of Machine Learning Research. PMLR, 2020. arXiv: 2002.02886 [cs.LG]. URL: https://arxiv.org/abs/2002.02886.

[40] Aleksandar Makelov, Georg Lange, and Neel Nanda. *Is This the Subspace You Are Looking for? An Interpretability Illusion for Subspace Activation Patching*. 2023. arXiv: 2311.17030 [cs.LG]. URL: https://arxiv.org/abs/2311.17030.

[41] Samuel Marks et al. *Auditing Language Models for Hidden Objectives*. 2025. arXiv: 2503.10965 [cs.AI]. URL: https://arxiv.org/abs/2503.10965.

[42] Anish Mudide et al. *Efficient Dictionary Learning with Switch Sparse Autoencoders*. 2025. arXiv: 2410.08201 [cs.LG]. URL: https://arxiv.org/abs/2410.08201.

[43] Neel Nanda et al. *A Pragmatic Vision for Interpretability*. 2025. URL: https://www.alignmentforum.org/posts/StENzDcD3kpfGJssR/a-pragmatic-vision-for-interpretability.

[44] Neel Nanda et al. "Progress measures for grokking via mechanistic interpretability". In: *International Conference on Learning Representations*. 2023. arXiv: 2301.05217 [cs.LG]. URL: https://arxiv.org/abs/2301.05217.

[45] Aaron van den Oord, Oriol Vinyals, and Koray Kavukcuoglu. "Neural Discrete Representation Learning". In: *Advances in Neural Information Processing Systems*. Vol. 30. 2017. arXiv: 1711.00937. URL: https://arxiv.org/abs/1711.00937.

[46] Kiho Park, Yo Joong Choe, and Victor Veitch. "The Linear Representation Hypothesis and the Geometry of Large Language Models". In: *Proceedings of the 41st International Conference on Machine Learning*. Vol. 235. Proceedings of Machine Learning Research. PMLR, 2024, pp. 39643–39666. arXiv: 2311.03658 [cs.CL]. URL: https://arxiv.org/abs/2311.03658.

[47] Kiho Park et al. "The Geometry of Categorical and Hierarchical Concepts in Large Language Models". In: *International Conference on Learning Representations*. 2025. arXiv: 2406.01506 [cs.CL]. URL: https://arxiv.org/abs/2406.01506.

[48] Gonçalo Paulo et al. "Automatically Interpreting Millions of Features in Large Language Models". In: *Proceedings of the 42nd International Conference on Machine Learning*. Vol. 267. Proceedings of Machine Learning Research. PMLR, 2025. arXiv: 2410.13928 [cs.LG]. URL: https://arxiv.org/abs/2410.13928.

[49] Michael T. Pearce et al. "Bilinear MLPs enable weight-based mechanistic interpretability". In: *International Conference on Learning Representations*. 2025. arXiv: 2410.08417 [cs.LG]. URL: https://arxiv.org/abs/2410.08417.

[50] Hamed Pirsiavash, Deva Ramanan, and Charless C. Fowlkes. "Bilinear classifiers for visual recognition". In: *Advances in Neural Information Processing Systems*. Vol. 22. 2009, pp. 1482–1490.

[51] Alec Radford et al. *Language Models are Unsupervised Multitask Learners*. Tech. rep. OpenAI, 2019.

[52] Abhay Sheshadri et al. *Why Do Some Language Models Fake Alignment While Others Don't?* 2025. arXiv: 2506.18032 [cs.LG]. URL: https://arxiv.org/abs/2506.18032.

[53] Andrew Wagenmaker et al. *Steering Your Diffusion Policy with Latent Space Reinforcement Learning*. 2025. arXiv: 2506.15799 [cs.RO]. URL: https://arxiv.org/abs/2506.15799.

[54] Junxuan Wang et al. *Towards Universality: Studying Mechanistic Similarity Across Language Model Architectures*. 2024. arXiv: 2410.06672 [cs.CL]. URL: https://arxiv.org/abs/2410.06672.

[55] James C. R. Whittington et al. "Disentanglement with Biological Constraints: A Theory of Functional Cell Types". In: *International Conference on Learning Representations*. 2023. arXiv: 2210.01768 [q-bio.NC]. URL: https://arxiv.org/abs/2210.01768.

[56] Zhengxuan Wu et al. "AxBench: Steering LLMs? Even Simple Baselines Outperform Sparse Autoencoders". In: *Proceedings of the 42nd International Conference on Machine Learning*. Vol. 267. Proceedings of Machine Learning Research. PMLR, 2025, pp. 67035–67080. arXiv: 2501.17148 [cs.CL]. URL: https://arxiv.org/abs/2501.17148.

[57] Zhen Xu et al. *Beyond Redundancy: Diverse and Specialized Multi-Expert Sparse Autoencoder*. 2025. arXiv: 2511.05745 [cs.LG]. URL: https://arxiv.org/abs/2511.05745.

[58] Neil Zeghidour et al. *SoundStream: An End-to-End Neural Audio Codec*. 2021. arXiv: 2107.03312 [cs.SD]. URL: https://arxiv.org/abs/2107.03312.

[59] Zhengyan Zhang et al. "MoEfication: Transformer Feed-forward Layers are Mixtures of Experts". In: *Findings of the Association for Computational Linguistics: ACL 2022*. Dublin, Ireland: Association for Computational Linguistics, 2022, pp. 877–890. DOI: 10.18653/v1/2022.findings-acl.71. URL: https://aclanthology.org/2022.findings-acl.71/.

[60] Roland S. Zimmermann et al. "Contrastive Learning Inverts the Data Generating Process". In: *Proceedings of the 38th International Conference on Machine Learning*. Vol. 139. Proceedings of Machine Learning Research. PMLR, 2021. arXiv: 2102.08850 [cs.LG]. URL: https://arxiv.org/abs/2102.08850.